

CMP1203 – Fundamentos de Programação Orientada a Objetos

Prof. Me Alexandre Cláudio de Almeida



**PUC
GOIÁS**

Aplicação CRUD

Construindo um sistema simples com
Java, JDBC e MySQL

Estrutura do Projeto



Modelo (Produto.java)

Define o que é um "Produto". É o nosso POJO (Plain Old Java Object) que carrega os dados.



DAO (ProdutoDAO.java)

Data Access Object. Contém toda a lógica de SQL (Create, Read, Update, Delete) separada.



Conexão (Conexao.java)

Uma classe utilitária responsável por abrir e fechar a conexão com o banco de dados MySQL.



Principal (Principal.java)

Nossa classe `main`. Ela usa o DAO para orquestrar as operações e testar a aplicação.

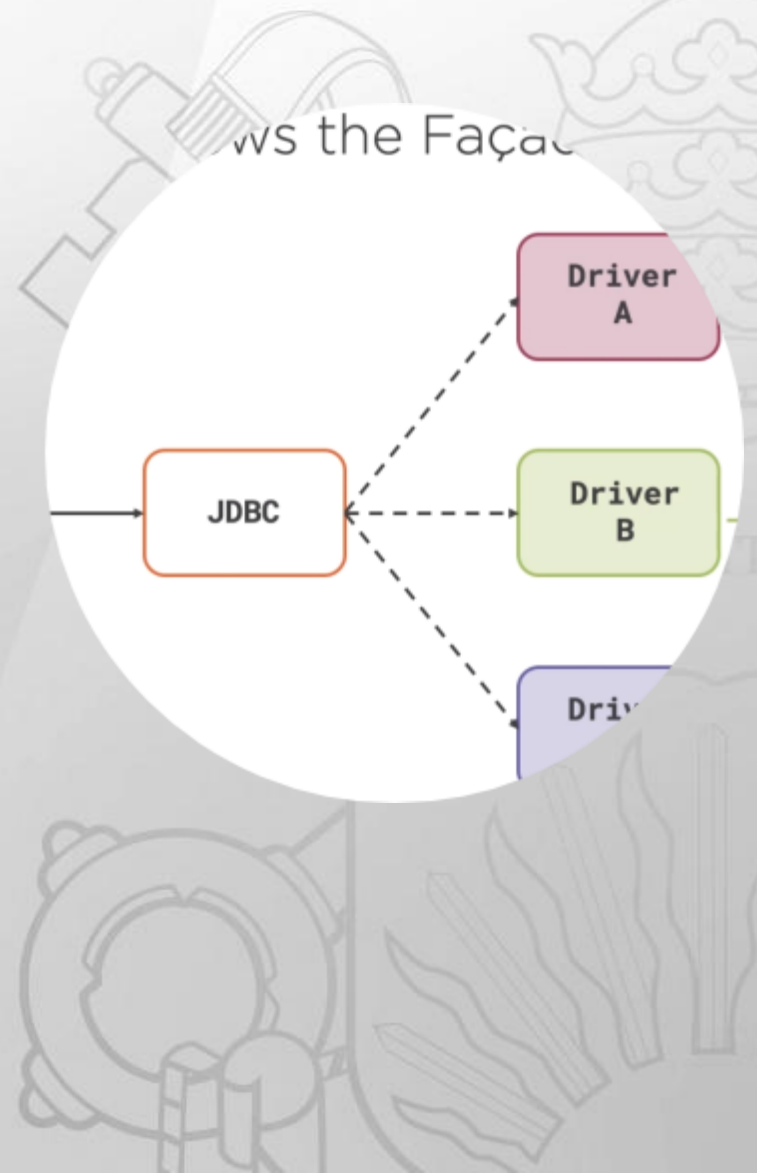
Configurando o Banco de Dados

- Primeiro, definimos a estrutura da nossa tabela no MySQL.
- CREATE DATABASE: Garante que o banco exista.
- USE: Seleciona o banco para uso.
- CREATE TABLE: Define a tabela produto com seus campos.
- id: Chave primária com auto-incremento.
- valor: Usamos DECIMAL(10, 2) para precisão monetária.

```
1  -- 1. Crie um banco de dados (se ainda não tiver)
2  • CREATE DATABASE IF NOT EXISTS bd_produtos;
3
4  -- 2. Use o banco de dados
5  • USE bd_produtos;
6
7  -- 3. Crie a tabela 'produto'
8  • CREATE TABLE IF NOT EXISTS produto (
9      id INT AUTO_INCREMENT PRIMARY KEY,
10     nome VARCHAR(255) NOT NULL,
11     quantidade INT,
12     valor DECIMAL(10, 2)
13 );
```

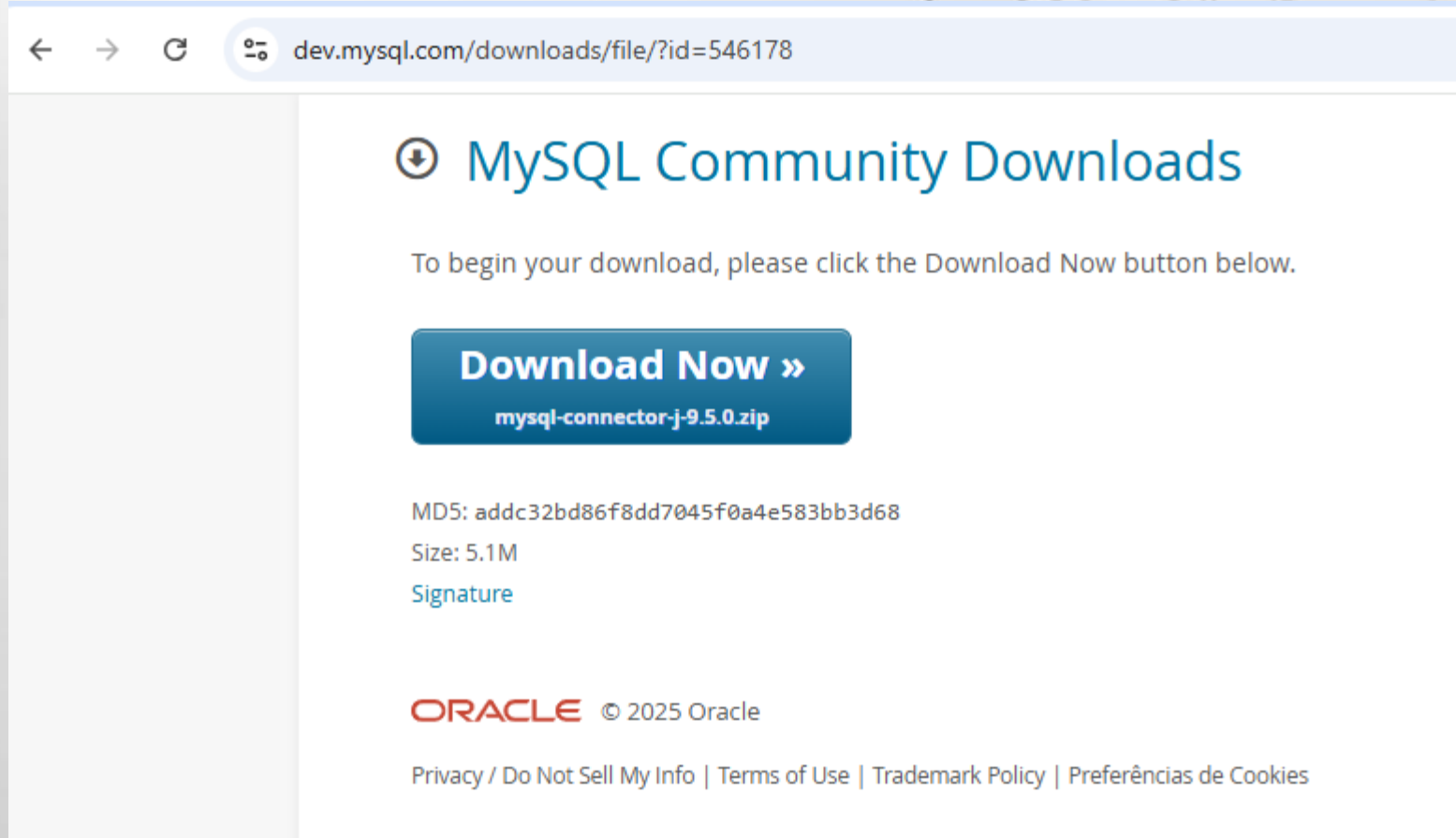
Driver JDBC

- O Java não conversa diretamente com o SGBD
- Ele precisa de um "tradutor" (driver) específico para cada SGBD.
- Precisamos baixar o arquivo mysql-connector-j-X.X.XX.jar manualmente.
- Este arquivo .jar deve ser adicionado ao "Build Path" (ou "Classpath") do nosso projeto na IDE (Eclipse, IntelliJ, etc.). Sem ele, o `Class.forName(...)` falhará.



Driver JDBC

- <https://dev.mysql.com/downloads/file/?id=546178>



The screenshot shows a web browser window with the address bar displaying `dev.mysql.com/downloads/file/?id=546178`. The page content includes the heading "MySQL Community Downloads" with a download icon, a instruction to click the "Download Now" button, and a large blue button labeled "Download Now »" with the filename `mysql-connector-j-9.5.0.zip` below it. Further down, the MD5 hash, size (5.1M), and a link to the signature are listed. The footer contains the Oracle logo, copyright notice, and links for privacy, terms, and cookies.

← → ↻ 🔍 dev.mysql.com/downloads/file/?id=546178

MySQL Community Downloads

To begin your download, please click the Download Now button below.

Download Now »
mysql-connector-j-9.5.0.zip

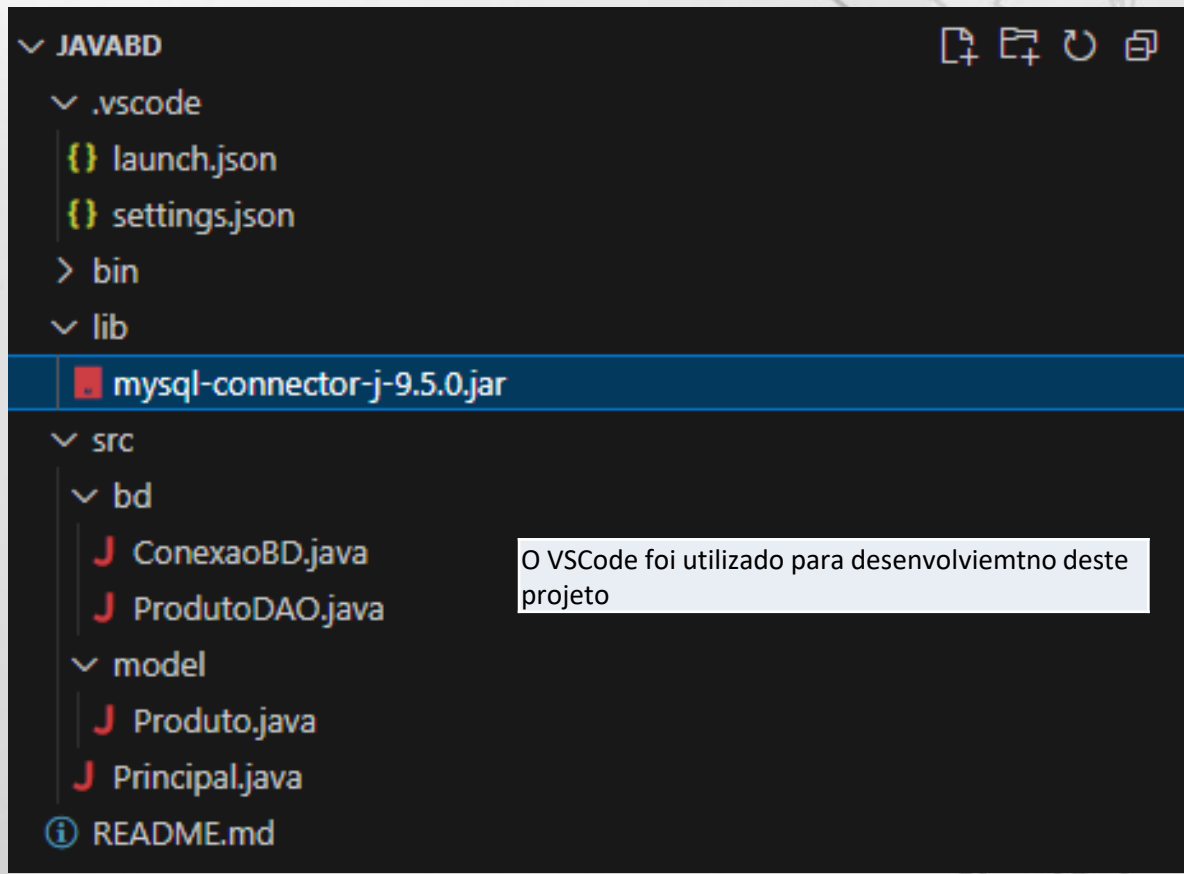
MD5: addc32bd86f8dd7045f0a4e583bb3d68
Size: 5.1M
[Signature](#)

ORACLE © 2025 Oracle

[Privacy](#) / [Do Not Sell My Info](#) | [Terms of Use](#) | [Trademark Policy](#) | [Preferências de Cookies](#)

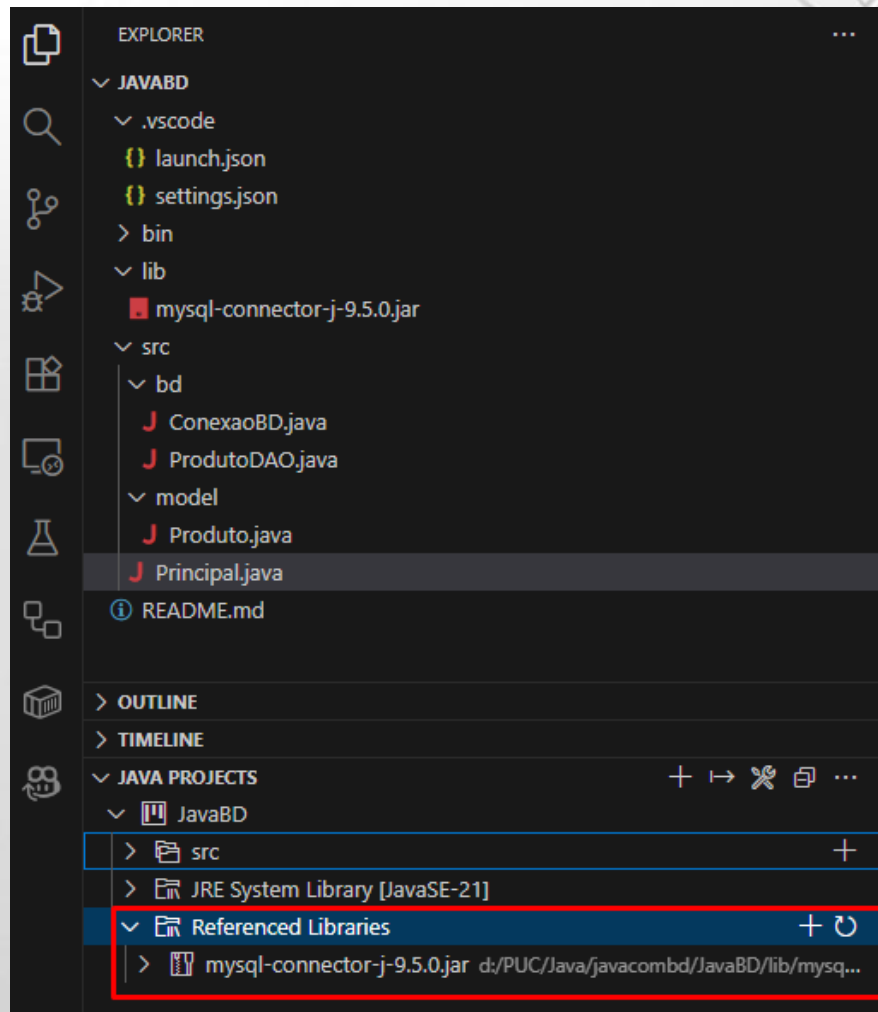
Driver JDBC

- Faça o download do arquivo e o coloque na pasta lib dentro do seu projeto



Driver JDBC

Confirme se o driver está disponível na seção ***Referenced Libraries***



O Modelo: Produto.java

- Esta é uma classe "POJO" (Plain Old Java Object). Ela não tem lógica complexa.
- Sua única responsabilidade é representar os dados de um produto em memória, servindo como um "contêiner" para transportar dados entre a aplicação e o banco de dados.
 - Atributos privados para encapsulamento.
 - Getters e Setters para acesso
 - BigDecimal é usado para dinheiro, evitando problemas de arredondamento do double.

O Modelo: Produto.java

```
J Produto.java X
src > model > J Produto.java > Produto > getId()
1  package model;
2
3  import java.math.BigDecimal;
4
5  /**
6   * POJO (Plain Old Java Object) que representa a entidade Produto.
7   */
8  public class Produto {
9
10     private int id;
11     private String nome;
12     private int quantidade;
13     private BigDecimal valor; // Usar BigDecimal para valores monetários
14
15     // Getters e Setters
16
17     public int getId() {
18         return id;
19     }
20
21     public void setId(int id) {
22         this.id = id;
23     }
24
25     public String getNome() {
26         return nome;
27     }
28 }
```

O Modelo: Produto.java

```
29 public void setNome(String nome) {
30     this.nome = nome;
31 }
32
33 public int getQuantidade() {
34     return quantidade;
35 }
36
37 public void setQuantidade(int quantidade) {
38     this.quantidade = quantidade;
39 }
40
41 public BigDecimal getValor() {
42     return valor;
43 }
44
45 public void setValor(BigDecimal valor) {
46     this.valor = valor;
47 }
48
49 // Sobrescrevendo o método toString() para facilitar a impressão
50 @Override
51 public String toString() {
52     return "Produto [ID=" + id + ", Nome=" + nome + ", Qtd=" + quantidade + ", Valor=R$ " + valor + "]\n";
53 }
54 }
```

Conexão: ConexaoBD.java

- Esta classe centraliza a lógica de conexão com o banco de dados.
 - Ela é fundamental para evitar repetição de código
 - URL: O "endereço" do banco (jdbc:mysql://localhost:3306/bd_produtos).
 - USUARIO/SENHA: Suas credenciais de acesso.
 - Class.forName(...): Carrega o driver JDBC (o .jar) que adicionamos manualmente.
 - DriverManager.getConnection(...): Tenta estabelecer a conexão.
- O método getConexao() é estático para que possamos chamá-lo de qualquer lugar (ex: ConexaoBD.getConexao()) sem precisar criar uma instância da classe.

Conexão: ConexaoBD.java

```
J ConexaoMySQL.java X
src > bd > J ConexaoMySQL.java > ConexaoMySQL
1  package bd;
2
3  import java.sql.Connection;
4  import java.sql.DriverManager;
5  import java.sql.SQLException;
6
7  /**
8   * Classe utilitária para gerenciar a conexão com o banco de dados MySQL.
9   */
10 public class ConexaoMySQL {
11     // URL de conexão JDBC
12     private static final String URL = "jdbc:mysql://localhost:3306/bd_produtos";
13     // Usuário do banco de dados
14     private static final String USUARIO = "root";
15     // Senha do banco de dados
16     private static final String SENHA = "123456";
17     // -----
18
19     /**
20      * Tenta estabelecer uma conexão com o banco de dados.
21      * @return um objeto Connection.
22      * @throws SQLException se houver um erro ao conectar.
23      */
24     public static Connection getConexao() throws SQLException {
25         try {
26             // Carrega o driver JDBC do MySQL
27             Class.forName(className: "com.mysql.cj.jdbc.Driver");
28             // Retorna a conexão estabelecida
29             return DriverManager.getConnection(URL, USUARIO, SENHA);
30         } catch (ClassNotFoundException e) {
31             // Lança uma exceção de SQL se o driver não for encontrado
32             throw new SQLException(reason: "Driver JDBC do MySQL não encontrado", e);
33         }
34     }
35 }
```

O DAO – ProdutoDAO.java

```
J ProdutoDAO.java X
src > bd > J ProdutoDAO.java > ...
1  package bd;
2
3  import model.Produto;
4
5  import java.sql.Connection;
6  import java.sql.PreparedStatement;
7  import java.sql.ResultSet;
8  import java.sql.SQLException;
9  import java.sql.Statement;
10 import java.util.ArrayList;
11 import java.util.List;
12
13 /**
14  * Data Access Object (DAO) para a entidade Produto.
15  * Contém todas as operações de CRUD.
16  */
17 public class ProdutoDAO {
18
```

ProdutoDAO.java - CREATE

```
22 public Produto create(Produto produto) throws SQLException {
23     // SQL para inserção
24     String sql = "INSERT INTO produto (nome, quantidade, valor) VALUES (?, ?, ?)";
25
26     // Try-with-resources para garantir que a conexão e o statement sejam fechados
27     try (Connection conn = ConexaoBD.getConexao());
28     {
29         PreparedStatement stmt = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS) {
30
31             // Define os parâmetros do PreparedStatement
32             stmt.setString(parameterIndex: 1, produto.getNome());
33             stmt.setInt(parameterIndex: 2, produto.getQuantidade());
34             stmt.setBigDecimal(parameterIndex: 3, produto.getValor());
35
36             // Executa a inserção
37             int affectedRows = stmt.executeUpdate();
38
39             if (affectedRows == 0) {
40                 throw new SQLException(reason: "Falha ao criar produto, nenhuma linha afetada.");
41             }
42
43             // Recupera o ID gerado pelo banco de dados
44             try (ResultSet generatedKeys = stmt.getGeneratedKeys()) {
45                 if (generatedKeys.next()) {
46                     produto.setId(generatedKeys.getInt(columnIndex: 1));
47                 } else {
48                     throw new SQLException(reason: "Falha ao criar produto, nenhum ID obtido.");
49                 }
50             }
51             return produto;
52         }
53     }
54 }
```

ProdutoDAO.java - CREATE

- O método create recebe um objeto Produto e o insere no banco.
- PreparedStatement: Usamos PreparedStatement (com ?) para evitar SQL Injection.
 - É a forma mais segura de executar queries.
- try-with-resources: O try (...) garante que Connection e PreparedStatement sejam fechados automaticamente, mesmo se ocorrer um erro.
- RETURN_GENERATED_KEYS: Pede ao banco para retornar o id que foi auto-incrementado, para que possamos atualizar nosso objeto Java.

ProdutoDAO.java - READ

```
54  /**
55   * READ (All) - Retorna todos os produtos do banco de dados.
56   */
57  public List<Produto> readAll() throws SQLException {
58      List<Produto> produtos = new ArrayList<>();
59      String sql = "SELECT * FROM produto";
60
61      try (Connection conn = ConexaoBD.getConexao();
62           PreparedStatement stmt = conn.prepareStatement(sql);
63           ResultSet rs = stmt.executeQuery()) {
64
65          // Itera sobre o ResultSet e cria objetos Produto
66          while (rs.next()) {
67              Produto produto = new Produto();
68              produto.setId(rs.getInt(columnLabel: "id"));
69              produto.setNome(rs.getString(columnLabel: "nome"));
70              produto.setQuantidade(rs.getInt(columnLabel: "quantidade"));
71              produto.setValor(rs.getBigDecimal(columnLabel: "valor"));
72              produtos.add(produto);
73          }
74      }
75      return produtos;
76  }
```

ProdutoDAO.java - READ

- O método `readAll` busca todos os registros da tabela `produto` e os retorna como uma lista de objetos `Produto`
- `ResultSet`: O `ResultSet (rs)` é um objeto que nos permite iterar sobre os resultados da consulta, linha por linha.
- Mapeamento: Dentro do `while (rs.next())`, nós "mapeamos" os dados de cada coluna do banco (ex: `rs.getString("nome")`) para os atributos de um novo objeto `Produto`.

ProdutoDAO.java – UPDATE e DELETE

```
78  /**
79  * UPDATE - Atualiza um produto existente no banco de dados.
80  */
81  public boolean update(Produto produto) throws SQLException {
82      String sql = "UPDATE produto SET nome = ?, quantidade = ?, valor = ? WHERE id = ?";
83
84      try (Connection conn = ConexaoBD.getConexao();
85           PreparedStatement stmt = conn.prepareStatement(sql)) {
86
87          stmt.setString(parameterIndex: 1, produto.getNome());
88          stmt.setInt(parameterIndex: 2, produto.getQuantidade());
89          stmt.setBigDecimal(parameterIndex: 3, produto.getValor());
90          stmt.setInt(parameterIndex: 4, produto.getId());
91
92          // Retorna true se 1 (ou mais) linha foi afetada
93          return stmt.executeUpdate() > 0;
94      }
95  }
```

```
97  /**
98  * DELETE - Remove um produto do banco de dados pelo ID.
99  */
100 public boolean delete(int id) throws SQLException {
101     String sql = "DELETE FROM produto WHERE id = ?";
102
103     try (Connection conn = ConexaoBD.getConexao();
104          PreparedStatement stmt = conn.prepareStatement(sql)) {
105
106         stmt.setInt(parameterIndex: 1, id);
107
108         // Retorna true se 1 (ou mais) linha foi afetada
109         return stmt.executeUpdate() > 0;
110     }
111 }
112 }
```

ProdutoDAO.java – UPDATE e DELETE

- **Update (Atualizar)**

- Recebe um objeto Produto e atualiza o registro no banco onde o id for correspondente.
- A ordem dos parâmetros (?) é crucial e deve corresponder à ordem dos stmt.set....

- **Delete (Remover)**

- Recebe um id e remove o registro correspondente do banco.
- Ambos os métodos retornam um boolean (baseado em `executeUpdate() > 0`), indicando se alguma linha foi de fato afetada pela operação.

A Execução – Principal.java

```
J Principal.java X
src > J Principal.java > ...
1  import bd.ProdutoDAO;
2  import model.Produto;
3
4  import java.math.BigDecimal;
5  import java.sql.SQLException;
6  import java.util.List;
7
8  /**
9   * Classe principal para demonstrar as operações de CRUD.
10  */
11  public class Principal {
12
13      Run | Debug
14      public static void main(String[] args) {
15          ProdutoDAO produtoDAO = new ProdutoDAO();
16
17          try {
18              // 1. CREATE (Criar)
19              System.out.println(x: "--- 1. Criando um novo produto ---");
20              Produto novoProduto = new Produto();
21              novoProduto.setNome(nome: "Notebook Gamer");
22              novoProduto.setQuantidade(quantidade: 10);
23              novoProduto.setValor(new BigDecimal(val: "7500.00"));
24
25              produtoDAO.create(novoProduto);
26              System.out.println("Produto criado com sucesso! ID: " + novoProduto.getId());
27              System.out.println(x: "-----\n");
28
29              // 2. READ (Ler Todos)
30              System.out.println(x: "--- 2. Lendo todos os produtos ---");
31              List<Produto> produtos = produtoDAO.readAll();
32              for (Produto p : produtos) {
33                  System.out.println(p);
34              }
35              System.out.println(x: "-----\n");
36          }
37      }
38  }
```

A Execução – Principal.java

```
38 // 3. UPDATE (Atualizar)
39 System.out.println("--- 3. Atualizando o produto (ID: " + novoProduto.getId() + ") ---");
40 // Vamos atualizar o produto que acabamos de criar
41 novoProduto.setValor(new BigDecimal(val: "7250.99")); // Dando um desconto
42 novoProduto.setQuantidade(quantidade: 8); // Vendeu 2
43
44 boolean atualizou = produtoDAO.update(novoProduto);
45 if (atualizou) {
46     System.out.println(x: "Produto atualizado com sucesso:");
47     // Imprime o estado atualizado (embora o DAO de readById fosse melhor aqui)
48     System.out.println(novoProduto);
49 } else {
50     System.out.println(x: "Falha ao atualizar o produto.");
51 }
52 System.out.println(x: "-----\n");
53
54
55 // 4. DELETE (Deletar)
56 System.out.println("--- 4. Deletando o produto (ID: " + novoProduto.getId() + ") ---");
57 boolean deletou = produtoDAO.delete(novoProduto.getId());
58 if (deletou) {
59     System.out.println(x: "Produto deletado com sucesso.");
60 } else {
61     System.out.println(x: "Falha ao deletar o produto.");
62 }
63 System.out.println(x: "-----\n");
64
65
66 // 5. READ (Verificação Final)
67 System.out.println(x: "--- 5. Lendo todos os produtos (após delete) ---");
68 List<Produto> produtosFinais = produtoDAO.readAll();
69 if (produtosFinais.isEmpty()) {
70     System.out.println(x: "Nenhum produto encontrado.");
71 } else {
72     for (Produto p : produtosFinais) {
73         System.out.println(p);
74     }
75 }
76 System.out.println(x: "-----\n");
77
78 } catch (SQLException e) {
79     System.err.println(x: "Ocorreu um erro de SQL:");
80     e.printStackTrace();
81 }
82 }
83 }
```

A Execução – Principal.java

- A classe Principal contém o método main e atua como orquestrador da nossa aplicação.
- Ela não contém nenhuma lógica de SQL. Sua única função é:
 - Instanciar o ProdutoDAO.
 - Criar objetos Produto
 - Chamar os métodos do DAO (create, readAll, update, delete) na ordem correta.
 - Imprimir os resultados no console.
- Essa separação de responsabilidades (main cuida do fluxo, DAO cuida do SQL) é a essência do padrão DAO.

A Execução – Principal.java

```
PS D:\PUC\Java\javacombd\JavaBD> d:; cd 'd:\PUC\Java\javacombd\JavaBD'; & 'C:\Program Files\Java\jdk-21\bin\java.exe'
'@C:\Users\Alexandre\AppData\Local\Temp\cp_brhh7jy6ioeqy57y2g1h8w1tj.argfile' 'Principal'
--- 1. Criando um novo produto ---
Produto criado com sucesso! ID: 1
-----

--- 2. Lendo todos os produtos ---
Produto [ID=1, Nome=Notebook Gamer, Qtd=10, Valor=R$ 7500.00]
-----

--- 3. Atualizando o produto (ID: 1) ---
Produto atualizado com sucesso:
Produto [ID=1, Nome=Notebook Gamer, Qtd=8, Valor=R$ 7250.99]
Produto atualizado com sucesso:
Produto [ID=1, Nome=Notebook Gamer, Qtd=8, Valor=R$ 7250.99]
-----

--- 4. Deletando o produto (ID: 1) ---
Produto deletado com sucesso.
-----

--- 5. Lendo todos os produtos (após delete) ---
Nenhum produto encontrado.
-----

PS D:\PUC\Java\javacombd\JavaBD>
```


Dúvidas?

